

Milestone 3 - Model Adaptation Experiment

Project: NLP-assisted job opportunity matching for MSBA international students

This notebook compares zero-shot prompting, few-shot prompting, and a true PEFT LoRA transformer on the same four-class job-posting triage task.

1. Data and Experimental Design

- Original public source rows: 785,741.
- Milestone 2 project sample: 100,000 balanced rows.
- Milestone 3 fixed validation set: 4,000 rows, 1,000 per label.
- Few-shot examples: 64 rows, 16 per label.
- PEFT LoRA training set: 2,000 rows, 500 per label.
- True LoRA base model: google/bert_uncased_L-2_H-128_A-2, run on cpu for 1 epoch.

Code cell 3

```
from pathlib import Path import json import pandas as pd DATA_PATH =  
Path('data_jobs_msba_project_sample_100k.csv') if not DATA_PATH.exists(): DATA_PATH =  
Path('data/data_jobs_msba_project_sample_100k.csv') RESULTS_PATH = Path('milestone3_adaptation_results.json') if  
not RESULTS_PATH.exists(): RESULTS_PATH = Path('data/milestone3_adaptation_results.json') df =  
pd.read_csv(DATA_PATH) results = json.loads(RESULTS_PATH.read_text()) print('Loaded rows:', len(df))  
print(pd.crosstab(df['relevance_label'], df['split']))
```

Output

```
Loaded rows: 100000 split train validation relevance_label high_fit 20000 5000 low_fit 20000 5000 medium_fit  
20000 5000 unclear 20000 5000
```

2. Strategy 1 - Zero-Shot Prompting

The zero-shot strategy uses only label definitions. In a hosted LLM setting, this would be a prompt with the four label descriptions and no examples. For a reproducible offline notebook, I implement the same idea as a transparent rubric classifier.

Code cell 5

```
zero = results['strategies']['zero_shot_prompt_rubric']['metrics'] print('Zero-shot accuracy:',  
round(zero['accuracy'], 4)) print('Zero-shot macro F1:', round(zero['macro_f1'], 4))
```

Output

```
Zero-shot accuracy: 0.8145 Zero-shot macro F1: 0.8147
```

3. Strategy 2 - Few-Shot Prompting

The few-shot strategy adds 16 examples per label. I treat those examples as in-context demonstrations by building class prototypes, which approximates how examples steer a prompt without requiring an external API.

Code cell 7

```
few = results['strategies']['few_shot_prompt_prototypes']['metrics'] print('Few-shot examples:',  
results['experiment_design']['few_shot_rows']) print('Few-shot accuracy:', round(few['accuracy'], 4))  
print('Few-shot macro F1:', round(few['macro_f1'], 4))
```

Output

```
Few-shot examples: 64 Few-shot accuracy: 0.8285 Few-shot macro F1: 0.8285
```

4. Strategy 3 - True PEFT LoRA Transformer

The third strategy is a full PEFT LoRA experiment using the transformer/PEFT stack. It trains query/value LoRA adapters on a small BERT model while leaving most base-model weights frozen, so it is the most faithful fine-tuning comparison in this milestone.

Code cell 9

```
lora = results['strategies']['true_peft_lora_transformer'] m = lora['metrics'] details = lora['peft_details']
print('Base model:', details['base_model']) print('PEFT LoRA train rows:', lora['training_rows'])
print('Validation rows:', results['experiment_design']['peft_lora_validation_rows']) print('LoRA rank:',
details['lora']['r']) print('Runtime seconds:', details['runtime_seconds']) print('LoRA accuracy:',
round(m['accuracy'], 4)) print('LoRA macro F1:', round(m['macro_f1'], 4))
```

Output

```
Base model: google/bert_uncased_L-2_H-128_A-2 PEFT LoRA train rows: 2000 Validation rows: 4000 LoRA rank: 8
Runtime seconds: 14.3 LoRA accuracy: 0.6442 LoRA macro F1: 0.6432
```

Adaptation finding. True PEFT LoRA is complete and reproducible, but in this CPU-friendly tiny-BERT run it underperforms the prompt/prototype strategies. That makes the comparison more useful: adaptation has a real cost and should wait until the team has stronger labels and a stronger training setup.

5. Results

Strategy	Adaptation data	Accuracy	Macro F1	Relative cost / effort
Zero-shot prompt rubric	0	0.815	0.815	Lowest - no training examples
Few-shot prototypes	64	0.829	0.829	Low - 64 examples
True PEFT LoRA transformer	2,000	0.644	0.643	High - transformer adapter training

6. 500-Word Analysis

For Milestone 3, I compared three adaptation strategies on the same MSBA job-posting triage task: zero-shot prompting, few-shot prompting, and a true PEFT LoRA transformer. The task is to classify real public job postings into `high_fit`, `medium_fit`, `low_fit`, or `unclear` for a graduate career advisor. I kept the Milestone 2 data plan: the source dataset has 785,741 records and the project sample has 100,000 balanced rows. For this experiment, every strategy was evaluated on the same 4,000-row validation set, with 1,000 examples per label. This common split matters because the comparison is about adaptation strategy, not sampling luck.

The zero-shot strategy uses only label definitions, like a prompt that tells the model what each fit level means. It is the cheapest and easiest approach because it does not need labeled examples or training time. Its result, 0.8145 accuracy and 0.8147 macro F1, is useful but should be interpreted carefully. The labels are weak labels created from transparent screening rules, so zero-shot can score well by matching rule language while still missing advisor judgment about whether a job is genuinely appropriate for an international MSBA student.

The few-shot strategy adds sixteen examples per label. In the reproducible offline notebook, I implemented this as prototype matching, which approximates how in-context examples steer a prompt. Few-shot improved to 0.8285 accuracy and 0.8285 macro F1 with only 64 examples. This fits the prompting readings because the model behavior changes through instructions and examples rather than parameter updates. It is also operationally attractive: a career advisor can inspect or replace examples without retraining a model.

The third strategy is true PEFT LoRA. I installed the transformer/PEFT stack and trained LoRA adapters on `google/bert_uncased_L-2_H-128_A-2` using 2,000 training rows and the same 4,000-row validation set. The adapter updated query and value modules with rank 8, alpha 16, and dropout 0.05 while leaving the base model mostly frozen. This is the most faithful fine-tuning experiment, but on a CPU-friendly tiny BERT model and one epoch it reached only 0.6443 accuracy and 0.6432 macro F1. That result is lower than prompting, but it is still valuable: it shows that true adaptation adds implementation cost and can underperform if the base model, training budget, or weak labels are not strong enough.

My recommendation is to use few-shot/prototype adaptation for the next milestone and treat PEFT LoRA as a later-stage option. Few-shot currently gives the best balance of quality, cost, and effort, while preserving transparency for advisor review. Operationally, that matters because a career-office tool must be easy for advisors to explain, correct, and monitor as hiring language changes across industries. LoRA should return after the team collects a smaller human-reviewed label set, especially examples involving ambiguous fit, sponsorship/CPT/OPT wording, and incomplete postings. The main project lesson is that adaptation is not automatically better than prompting; it becomes worthwhile only when the data quality and training setup justify the extra complexity.